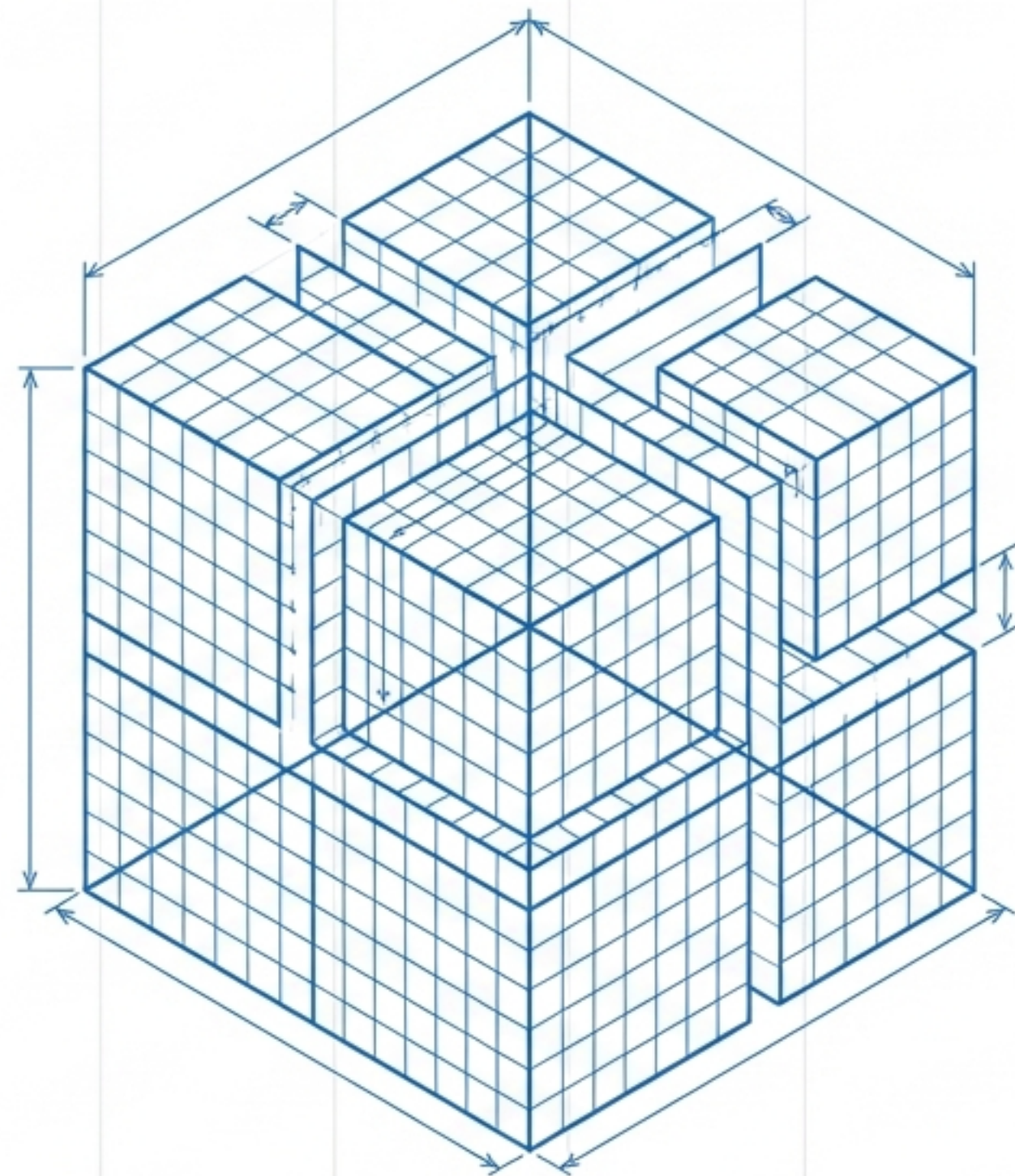


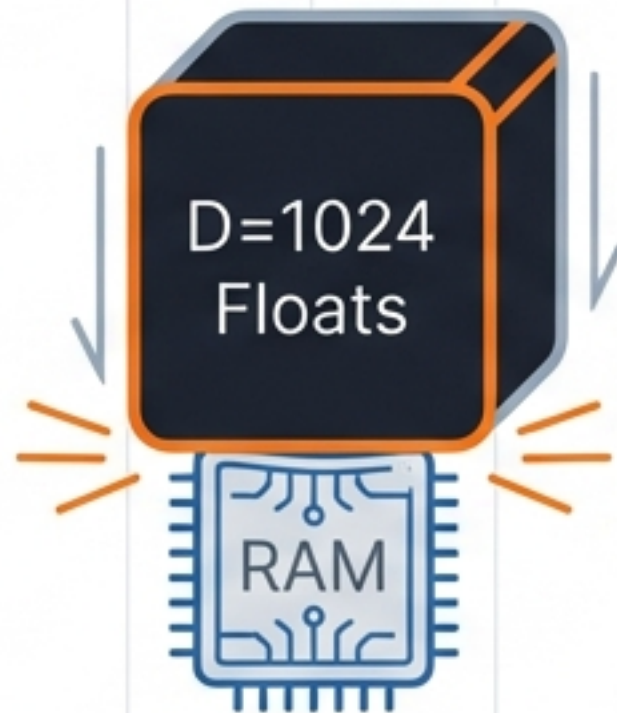
Advanced Vector Indexing: Scaling Beyond RAM

Navigating Sharding, Quantization, and Disk-Resident Graphs



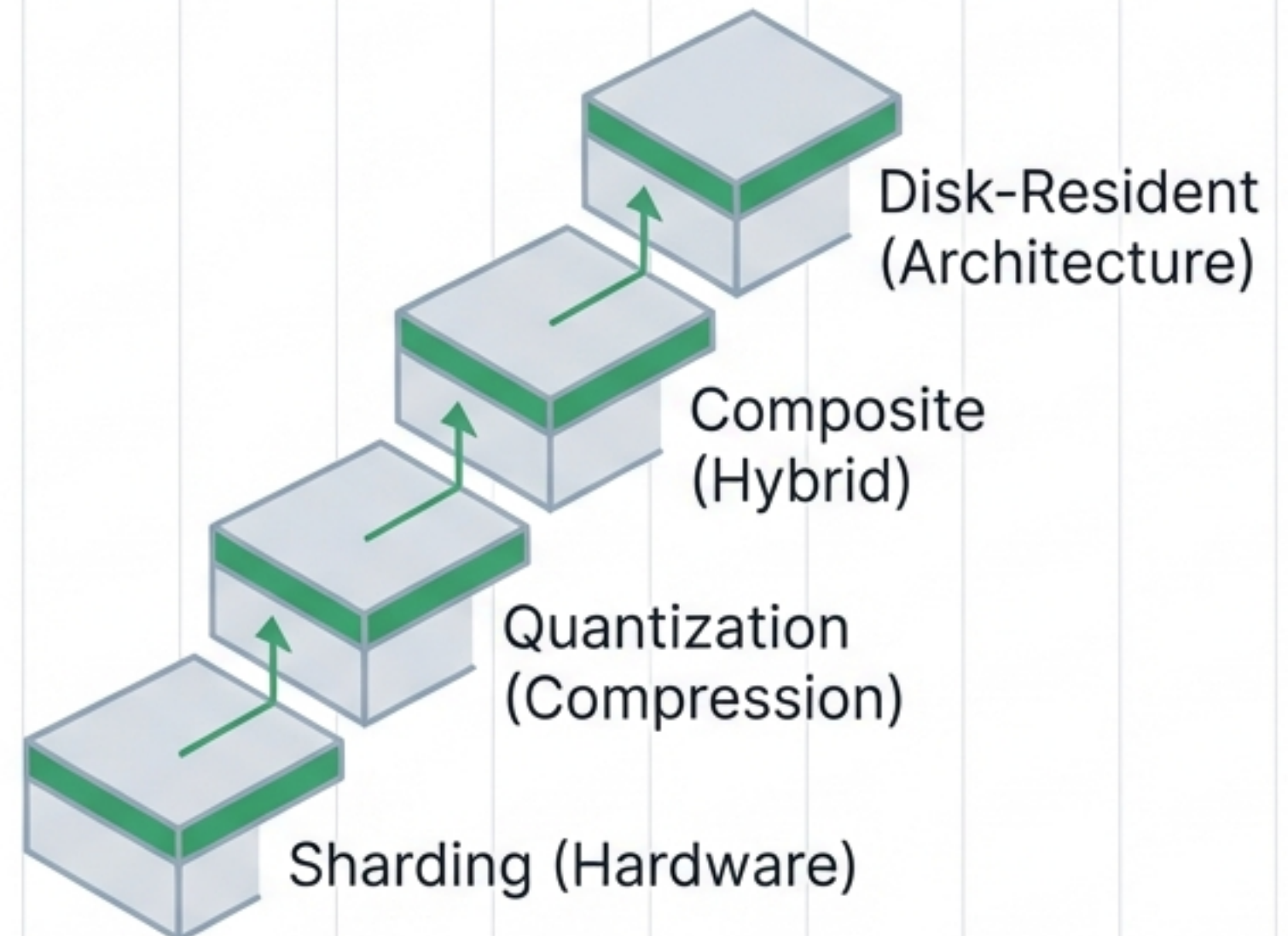
The scalability bottleneck is memory, not just search speed.

The Memory Barrier

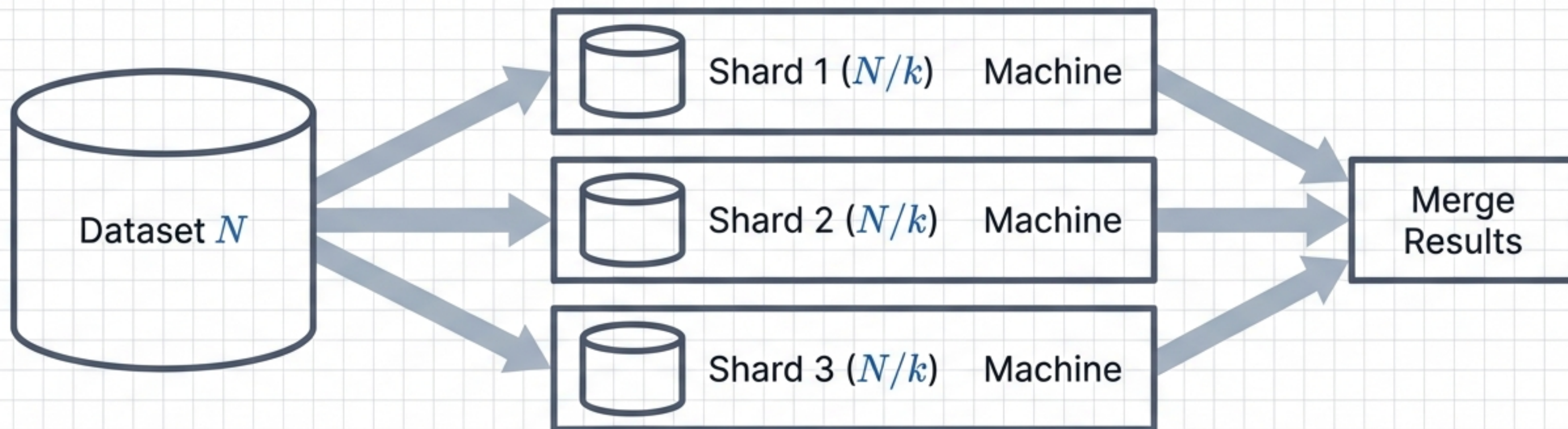


- Vectors are heavy ($D=1024+\$$).
- **Indexes** grow linearly with N .
- Rebuilds are expensive.

Scaling Ladder



Solution 1: Sharding (The Brute Force Approach)



Pros

- Overcomes single-machine RAM limit.
- Parallel search & insert.

The Trade-off

- Cost increases linearly (k machines).
- Latency = Slowest Shard.
- Network overhead.
- **Verdict: Delays the memory problem, doesn't solve it.**

Shrinking the Data: Scalar vs. Vector Quantization

Scalar Quantization (SQ)

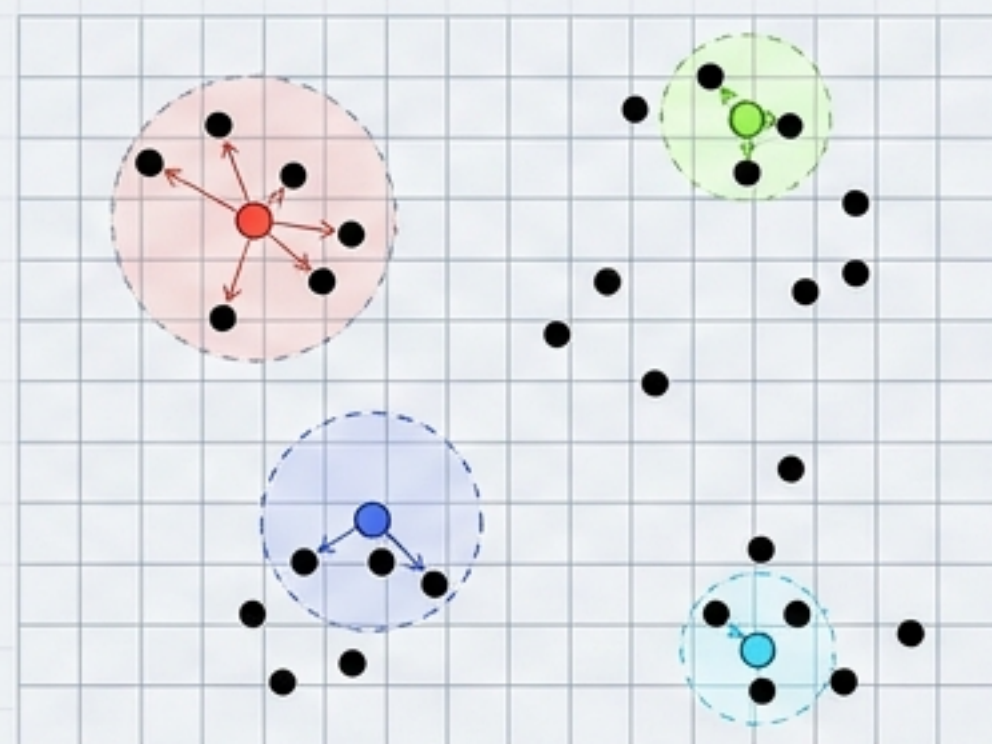
Reduce precision per component
(Float32 $\rightarrow$ Int8).

3.1415926535... $\rightarrow$ 3

Note: 4x smaller, ~1% recall loss.

Vector Quantization (VQ)

Cluster vectors into a codebook.

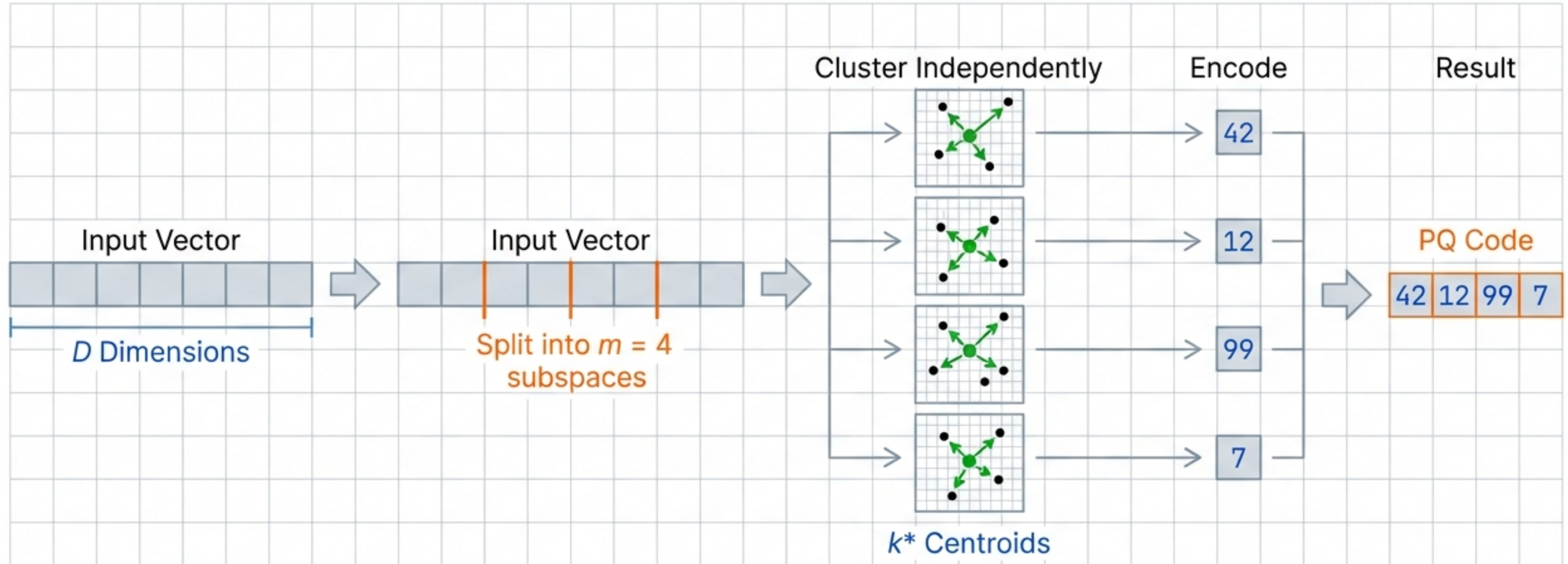


The VQ Problem

To minimize error in high dimensions (D), the number of clusters (k) must grow exponentially. Large codebooks become too slow to search ($O(DNk)$) and too big to store.

The Breakthrough: Product Quantization (PQ)

Divide and conquer the vector space.



Instead of one giant codebook (k), we use m small codebooks (k^*).
This provides exponential representational power with linear storage.

PQ: Massive Compression, Significant Trade-off

Raw Vectors (FP32)

Storage: 4096 bits/vector

Complexity: Exact calculation.

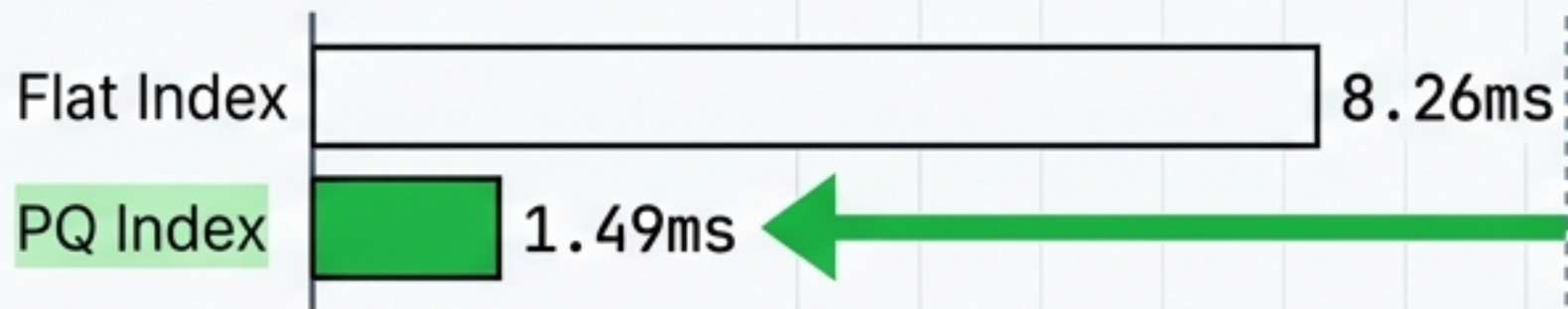
Product Quantization (PQ)

Storage: 64 bits/vector

Complexity: Look-up tables.

Performance Reality

Latency



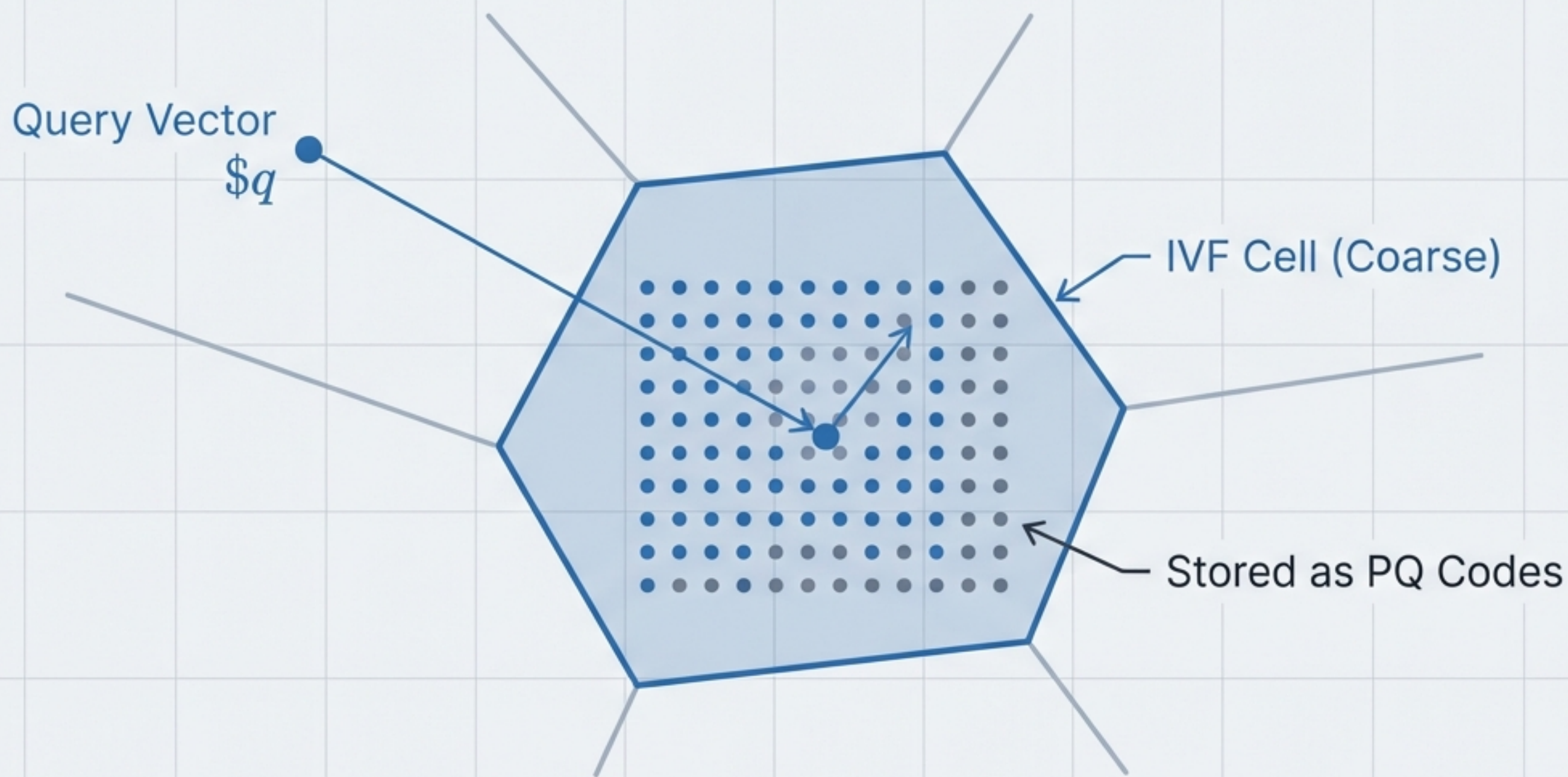
Recall



PQ solves memory and speed, but sacrifices accuracy. It needs a partner.

Composite Indexing: IVF + PQ

Combine a coarse router (IVF) to narrow the search, and a fine encoder (PQ) to compress the data.



Memory: 9MB (vs 512MB)



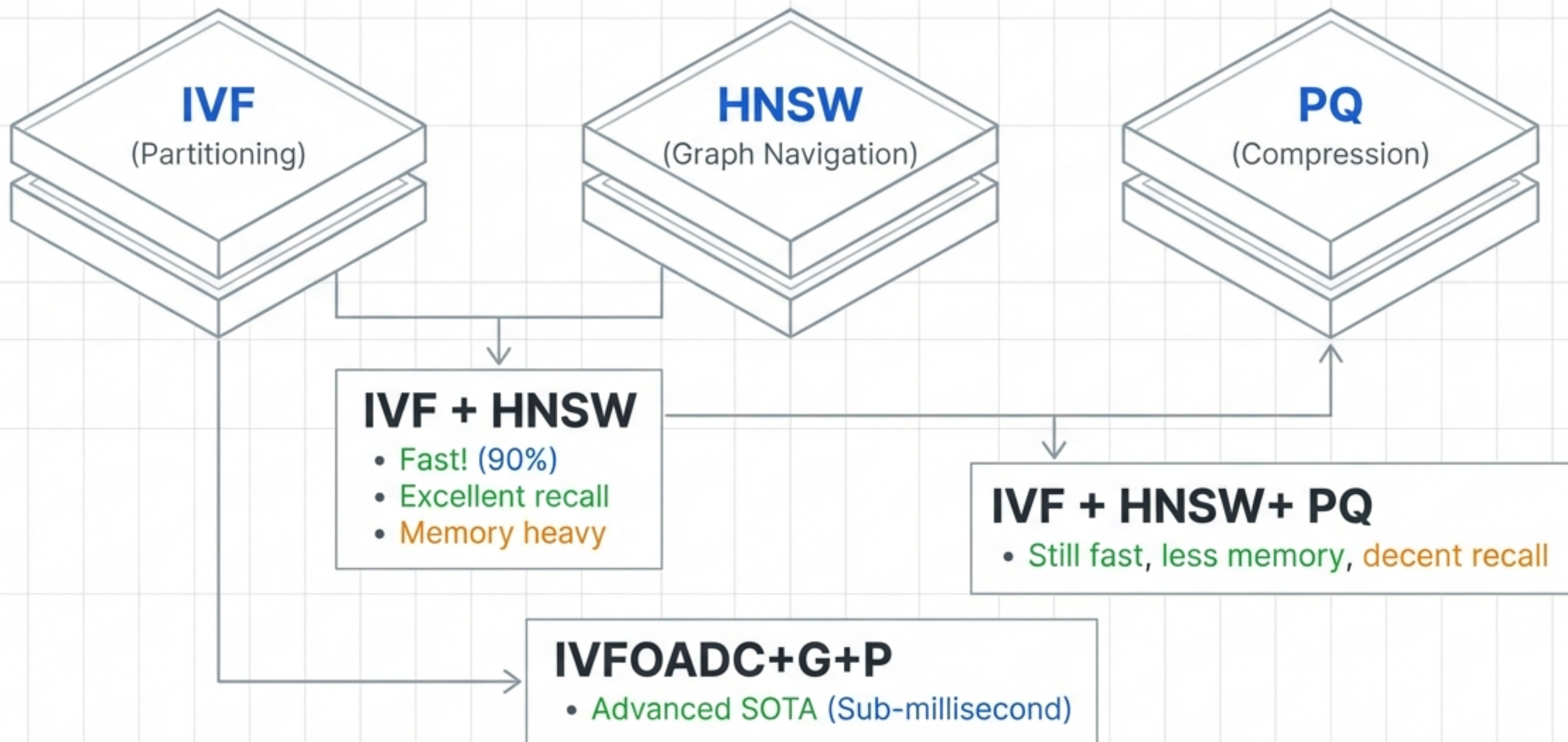
Latency: 0.09ms (Ultra Fast)



Recall: 52% (Still low)

The Composite “Cinematic Universe”

Combining components for State-of-the-Art results.



Crossing the RAM Barrier: Disk-Resident Indexes

This slide marks a major section transition.

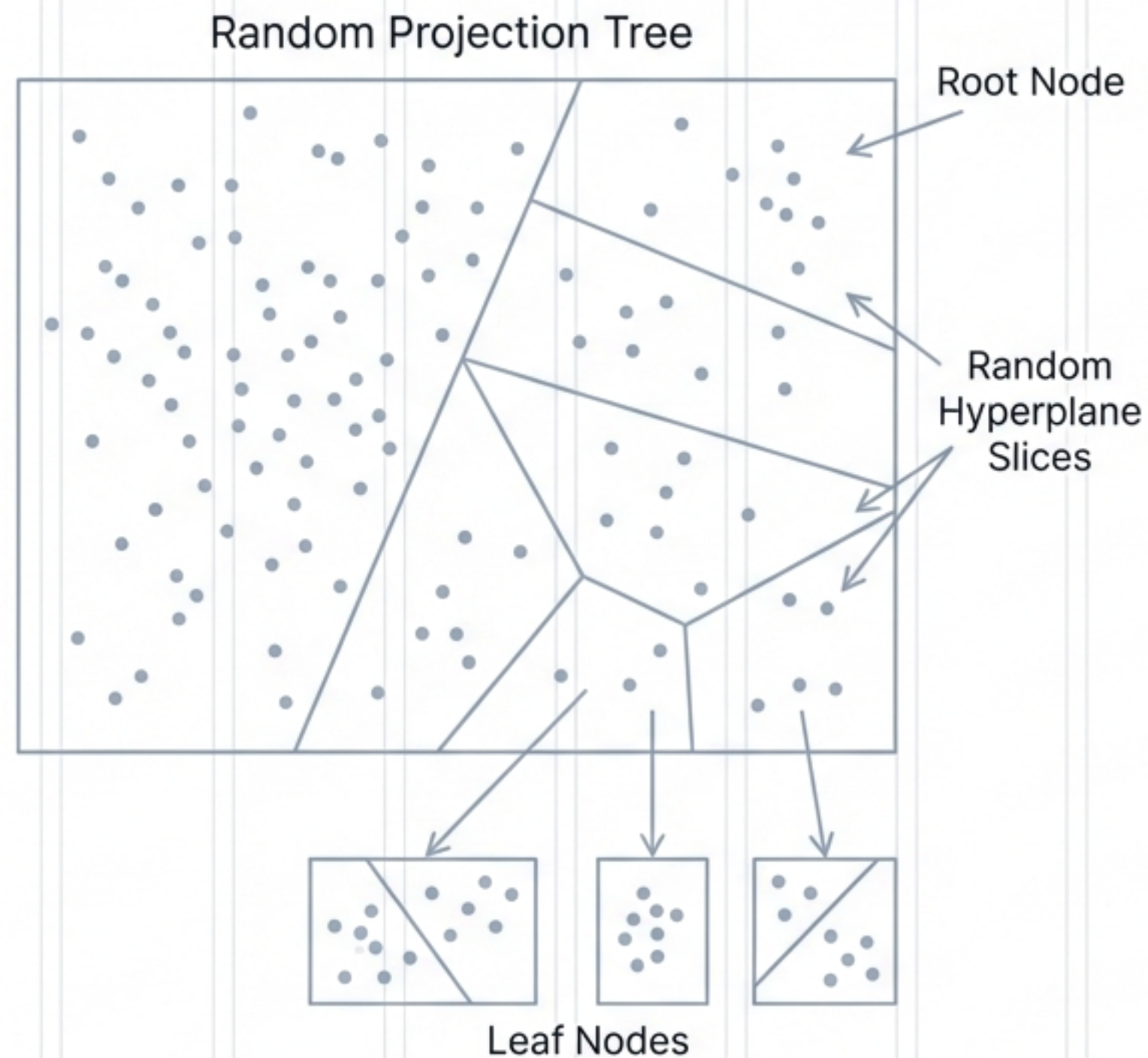


Challenge:

SSDs excel at sequential reads but struggle with random IOPs. Traditional graphs like HNSW require too many random "hops", killing performance. We need a new architecture.

The Static Approach: ANNOY

Tree-based indexing on disk (used by Spotify).



- **Mechanism:** Recursive splitting via random hyperplanes.
- **Disk Strategy:** Uses `mmap` to map file to virtual memory. Relies on OS `page cache`.
- **Pros:** Fast load/unload, great for static data.
- **Cons:** Read-only (no updates), higher memory footprint than graphs.

DiskANN: Reimagining Graphs for SSDs

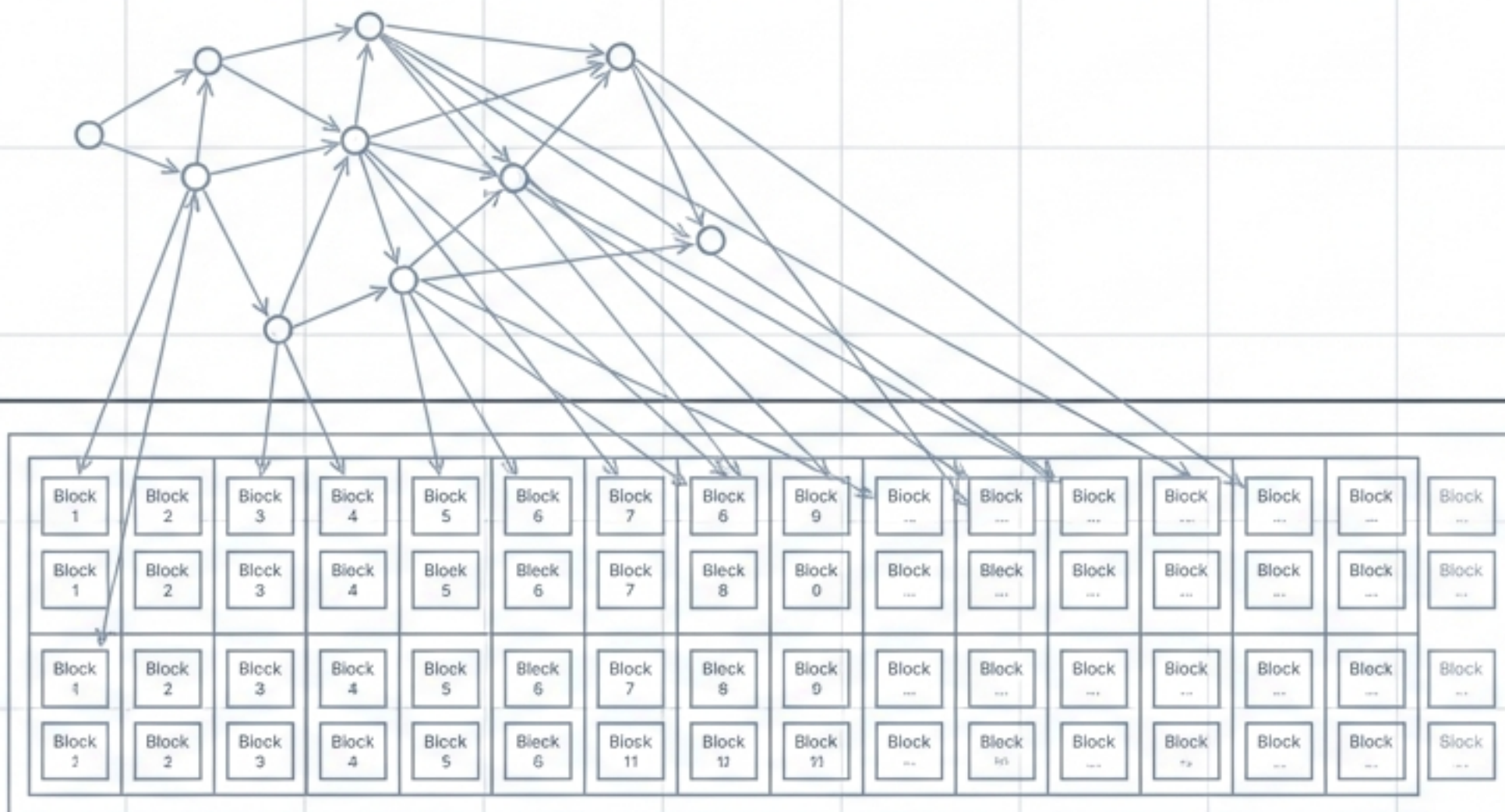
Split Architecture

RAM (Fast, Expensive)

Compressed Vectors (PQ)
for fast distance estimation.

SSD (High Latency, Cheap)

Full Precision Vectors +
Neighbor Lists.



Strategy: Keep the navigation skeleton in RAM, store the heavy data on Disk. Minimize hops to reduce I/O.

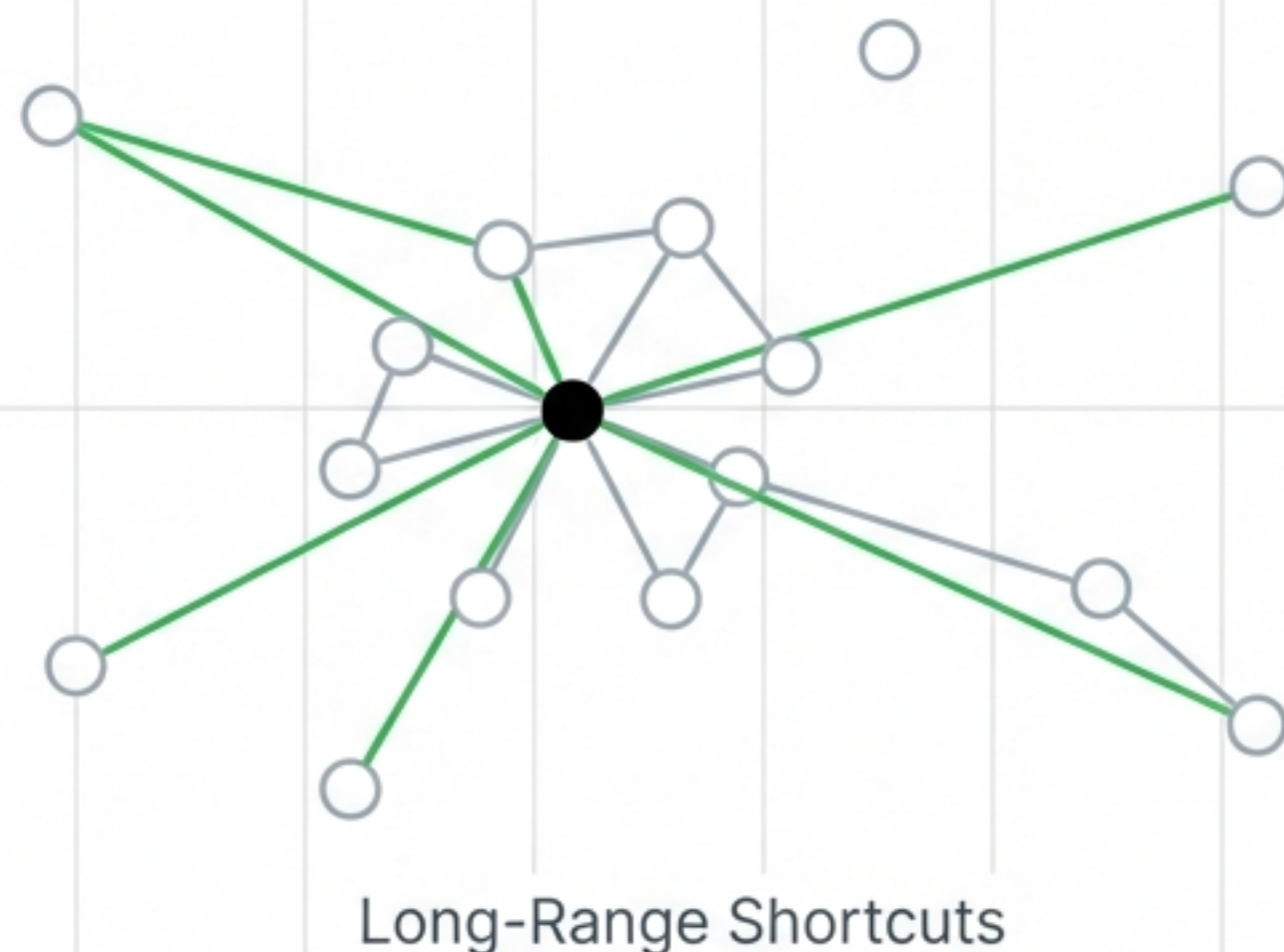
Graph Construction: The Vamana Algorithm

Creating 'Long-Range Highways' to minimize disk hops.

Standard Pruning



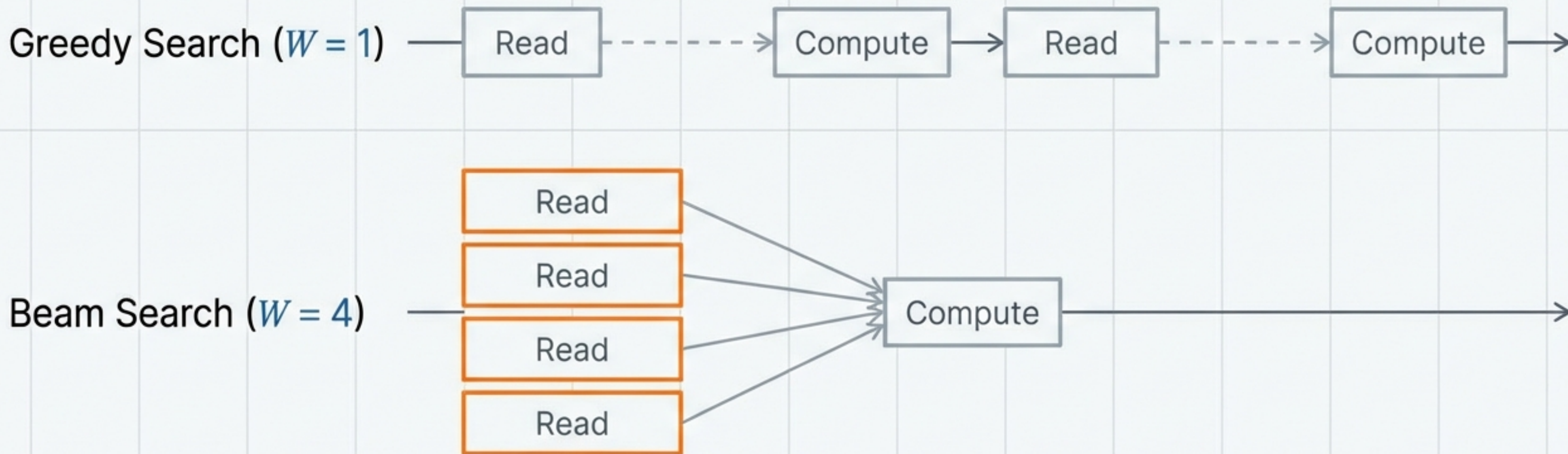
Vamana Pruning ($\alpha > 1$)



Standard graphs prune neighbors that are already "covered". Vamana relaxes this with an alpha parameter (α), intentionally stretching edges. This allows the search to traverse the dataset in fewer hops, minimizing disk reads.

Query Optimization: Beam Search

Greedy search (1 node at a time) leaves SSD bandwidth unused.
Beam search explores W nodes in parallel.

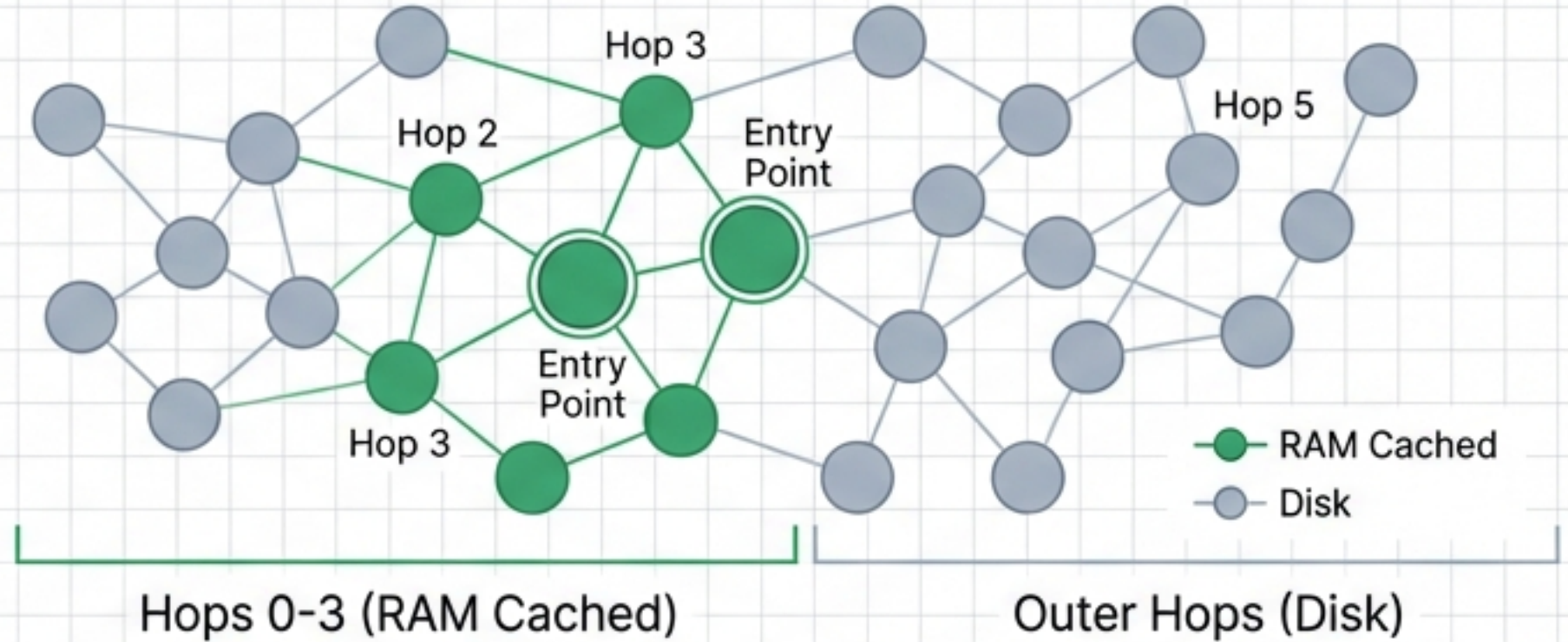


Sweet spot: $W \in [2, 4, 8]$ saturates I/O without spiking latency.

Final Optimizations: Caching & Re-ranking

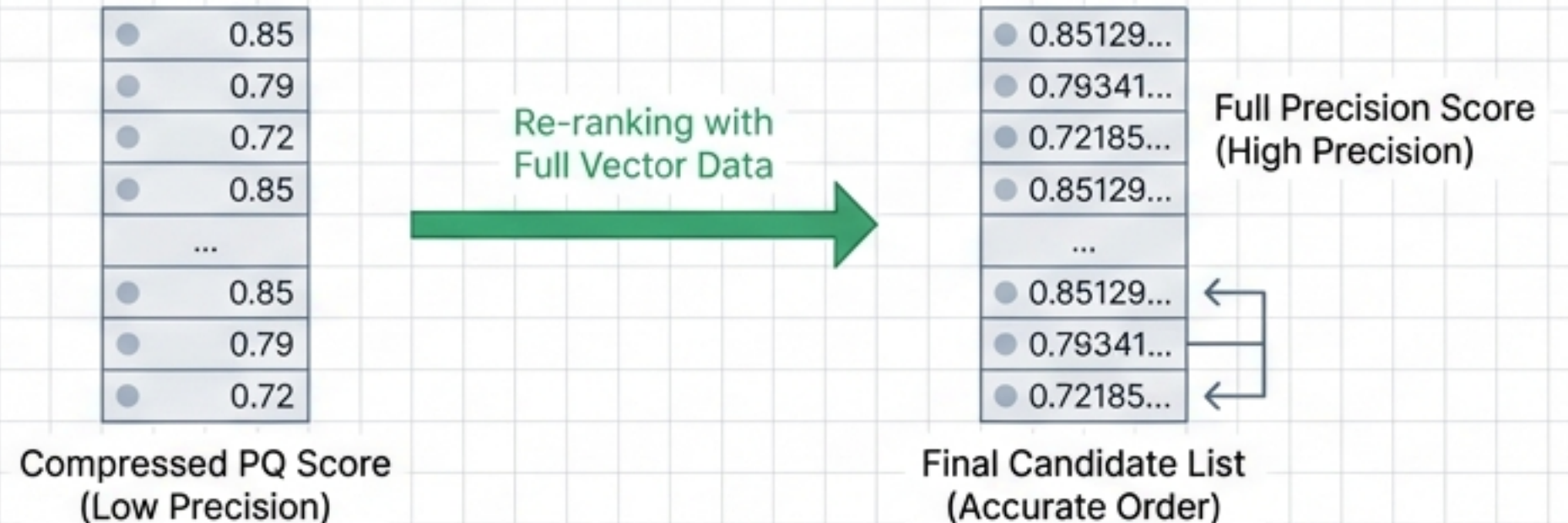
Node Caching

Hops 0-3 are accessed constantly.
Caching them in RAM saves expensive
initial disk reads.



Implicit Re-ranking

Since we fetch the full vector from
disk to find neighbors, we get
the high-precision data 'for free'.
We use this to re-score
candidates for perfect accuracy.



The Engineer's Decision Matrix

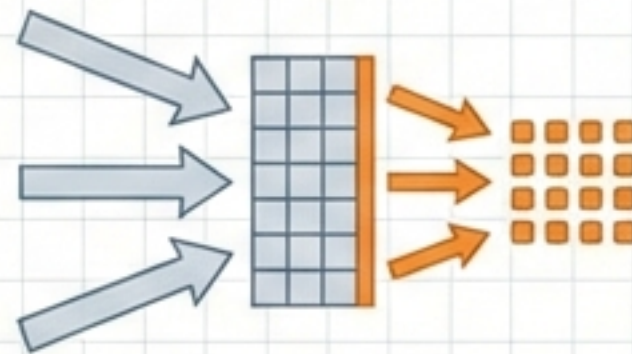
Choose Your Architecture

Sharding



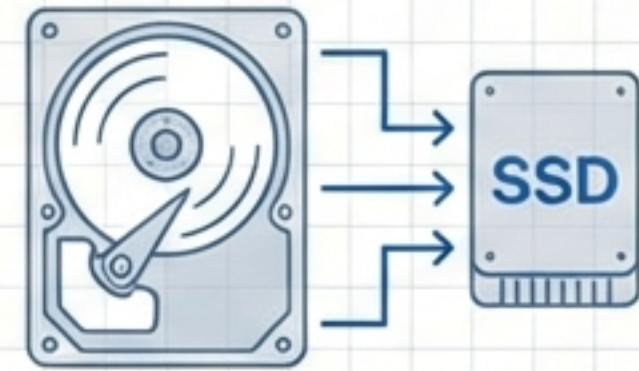
- Use when: High budget
- Low Latency requirement
- <100M vectors.

Composite (IVF+PQ)



- Use when: RAM constrained
- moderate accuracy loss is acceptable.

DiskANN



- Use when: Massive Scale (1B+)
- SSDs available
- cost-efficiency is key.

The 'perfect' index does not exist. It is a composite of your constraints: Memory vs. Accuracy vs. Scale.